

Transforma textos en embeddings

Informe de laboratorio

Harold Daniel Duque / CC 1033726983 / 6 de septiembre de 2026

Resumen

Cuando empecé el laboratorio, la pregunta que más me quedó fue esta: ¿cómo hace un programa para comparar dos textos sin quedarse solamente en las palabras que coinciden? El punto de partida son los *embeddings*: representaciones numéricas que recogen parte del contexto de las palabras o de las oraciones. Eso no significa que el computador entienda el lenguaje como una persona, pero sí le permite calcular cercanías, agrupar textos y buscar documentos relacionados.

El desarrollo completo está en el notebook `laboratorio_embeddings.ipynb`.

1. Objetivo y planteamiento

La idea fue armar un laboratorio que no se quedara en convertir palabras a números y ya. Para eso usé tickets simulados de servicio al cliente de una tienda virtual. Primero normalicé y tokenicé solicitudes sobre entregas, pagos y acceso a cuenta. Después construí una capa `Embedding` de Keras para mirar cómo se representan las palabras dentro de un clasificador. Por último comparé oraciones completas con embeddings preentrenados y similitud coseno. Me pareció útil dejar las dos aproximaciones porque una muestra el mecanismo por dentro y la otra responde mejor cuando aparece texto nuevo o una palabra que no estaba en un vocabulario pequeño.

La parte aplicada usa 60 tickets sintéticos. Hay 30 casos semilla con un tema de referencia, 15 casos separados para validación y 15 solicitudes operativas sin tema. Los primeros representan cómo podría verse una revisión humana dentro de una empresa; no son conversaciones reales ni estoy afirmando que alguien las haya revisado fuera de este ejercicio. La separación importa: si el modelo ve todo antes de evaluarse, una métrica bonita puede ser solo memoria. Los mensajes incluyen casos cotidianos como rastreos detenidos, paquetes incompletos, cobros duplicados, reembolsos pendientes y bloqueos de cuenta. También dejé una frase con *passkey* para mostrar que un vocabulario local puede mandar un término poco frecuente al token desconocido.

2. De texto a vector

El procesamiento empieza con `TextVectorization`. La capa convierte cada cadena en una secuencia de índices enteros: baja las mayúsculas, separa tokens y asigna un número a cada elemento del vocabulario. TensorFlow documenta esta capa como un mecanismo para llevar texto a secuencias de enteros que otra capa de red puede usar (TensorFlow, s. f.). Esta etapa no entrega todavía un significado; organiza la entrada para que el modelo la reciba.

Después aparece la capa `Embedding`. Si el vocabulario tiene V tokens y se define una dimensión d , la capa aprende una matriz $E \in \mathbb{R}^{V \times d}$. Cuando entra el índice de una palabra, el modelo consulta la fila correspondiente. Al comienzo esos valores no tienen una interpretación clara; durante el entrenamiento se ajustan según la tarea. La intuición es que palabras que ayudan a resolver contextos parecidos terminan ubicándose en zonas cercanas del espacio vectorial. Esta representación distribuida evita tratar cada palabra como una etiqueta aislada (Mikolov et al., 2013).

Para frases completas, el laboratorio usa `paraphrase-multilingual-MiniLM-L12-v2` mediante

Sentence Transformers. El modelo produce un vector por oración y luego normaliza esos vectores. La comparación se hace con la similitud coseno:

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}.$$

Un valor cercano a 1 indica direcciones similares; un valor bajo indica poca cercanía para ese modelo. No uso un umbral universal como “0,80 significa que dos frases son iguales”. El puntaje depende del modelo, del idioma y de la tarea. Lo útil es comparar el orden relativo de los resultados y revisar ejemplos que tengan sentido en el dominio.

3. Diseño del laboratorio

El notebook conserva la parte corta de tokenización y del embedding aprendido con Keras, pero el flujo principal se parece más al que podría usarse para ordenar solicitudes de una empresa. Los 30 tickets semilla tienen etiquetas de referencia: entregas, pagos o cuenta. Con sus embeddings se ajusta una regresión logística, un clasificador sencillo que recibe vectores y aprende a separar temas. Usé las implementaciones de *scikit-learn* (Pedregosa et al., 2011).

Los 15 tickets de validación se mantienen aparte. Después del entrenamiento se calcula el reporte de clasificación y la matriz de confusión; así se puede detectar, por ejemplo, si el sistema confunde reembolsos con novedades de entrega. Los otros 15 tickets entran sin etiqueta. El programa propone un tema, muestra la confianza y calcula la distancia entre sus dos mejores opciones. Si la confianza es menor que 0,70 o las dos opciones quedan demasiado parejas, el registro se pinta de rojo y queda como “Revisión humana”.

También se usa **KMeans**. Este algoritmo recibe los embeddings sin ver los nombres de las categorías y forma tres grupos por cercanía. Después se comparan los grupos con los tickets de referencia para entender qué parecen reunir. El algoritmo propone agrupaciones; ponerles un nombre y decidir la ruta sigue siendo trabajo de una persona.

El 0,70 y el margen de 0,18 son reglas de demostración, no números mágicos. En una operación real los escogería revisando el histórico, la proporción de casos que el equipo puede revisar y, sobre todo, el costo de mandar un ticket al área equivocada. Por la misma razón, el clasificador de este informe no se presenta como un producto terminado: la muestra es pequeña y sintética.

Evidencia	Qué se revisa al ejecutar el notebook
Tokenización	Vocabulario, secuencias de índices y token para elementos desconocidos.
Embedding local	Forma de la matriz de embeddings y flujo de entrenamiento del clasificador.
Similitud semántica	Matriz coseno, mapa de calor, paráfrasis y contrastes no relacionados.
Validación separada	Reporte y matriz de confusión sobre 15 tickets que no se usaron para ajustar el clasificador.
Revisión de incertidumbre	Tabla operativa, confianza, margen y registros rojos que deben verificarse.
Agrupamiento	Tabla de grupos y ejemplos que permiten discutir la categoría sugerida.
Persistencia	CSV de similitudes, CSV de clasificación operativa y CSV de agrupamiento.

4. Lectura de los resultados

Al ejecutar el notebook, uno de los pares que debería aparecer entre las similitudes más altas es “El pedido no se mueve desde hace tres días en el seguimiento” y “El rastreo de mi compra sigue igual y no registra avances”. No repiten exactamente las mismas palabras, pero los dos tickets describen el mismo problema: el cliente no sabe qué pasó con su envío. Ese caso permite ver por qué un embedding de oración es más útil que una coincidencia literal de términos.

La prueba de validación responde una pregunta diferente: con ejemplos que no vio durante el ajuste, ¿el clasificador mantiene separadas las tres rutas? El notebook no fija una exactitud de antemano; imprime lo que resulte en la ejecución. Esa distinción me parece importante. Con apenas 30 ejemplos semilla, un resultado alto es una señal para seguir evaluando, no permiso para automatizar todas las decisiones.

En el lote operativo, el tema propuesto no es el resultado final del proceso. Un ticket como “El pedido cambió de dirección y ahora el pago no aparece” mezcla dos asuntos y es buen candidato para tener poca seguridad o un margen reducido. Si queda rojo, no se fuerza una categoría: se envía a una persona que puede leer el caso, escoger una ruta o crear un tema nuevo si hace falta. Luego esa corrección podría sumarse a un lote revisado para actualizar el modelo. Ahí es donde el embedding sirve como insumo: convierte el texto a un vector que el clasificador y el agrupador pueden comparar; no reemplaza la decisión de negocio.

El agrupamiento completa la vista supervisada. Si varios tickets sin etiqueta caen cerca de los casos de rastreo, el grupo puede sugerir una categoría de entregas. Si aparece un grupo mezclado, no hay que disfrazarlo como un éxito: puede indicar mensajes ambiguos, pocos ejemplos o que las tres categorías son demasiado generales. La tabla cruzada del notebook ayuda a mirar esa mezcla y la gráfica 3D permite ubicar los puntos rojos frente a los demás. La gráfica es una proyección de muchas dimensiones a tres ejes, así que sirve para explorar, no para medir con exactitud la distancia semántica.

Los archivos `resultados_similitud.csv`, `resultados_clasificacion_operativa.csv` y `resultados_agrupamiento.csv`, creados al final de las secciones respectivas, conservan lo que produjo cada ejecución. De esa forma los resultados no quedan solo en una explicación escrita.

5. Conclusiones

Al hacer el ejercicio pude separar dos cosas que al comienzo me parecían casi iguales. Tokenizar pasa el texto a índices; un embedding aprende o recupera una representación densa para trabajar con esos índices. Lo primero organiza la entrada. Lo segundo intenta conservar relaciones que le sirvan al modelo para una tarea.

También confirmé que un embedding no elimina la revisión humana. Puede acelerar el triage: propone un tema, agrupa casos parecidos y muestra dónde la decisión es incierta. Pero si un ticket tiene información mezclada, o si el modelo está poco seguro, lo razonable es marcarlo para revisión y usar la corrección posterior como nuevo dato de referencia. Ese ciclo parece más útil y más honesto que tratar una probabilidad como si fuera una respuesta definitiva.

Referencias

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space*. Proceedings of the International Conference on Learning Representations. <https://arxiv.org/abs/1301.3781>

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Proces-*

sing, 3982-3992. <https://doi.org/10.18653/v1/D19-1410>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://jmlr.org/papers/v12/pedregosa11a.html>

TensorFlow. (s. f.). *tf.keras.layers.TextVectorization*. Recuperado el 6 de septiembre de 2026, de https://www.tensorflow.org/api_docs/python/tf/keras/layers/TextVectorization